Problem Set 3.1: The Sensor Anomaly Filter

Coding for AI - Module 1, Week 3: Sequence Manipulation

Rohit Kumar Singh
ID: `A26MTAI0004`

1. Logic Trace: Empty Batches

When a Python `for` loop encounters an empty sequence (like an empty batch list `[]`), it evaluates the length (which is 0) and immediately bypasses the inner loop body. Because Python's iteration relies on the iterable's actual contents rather than a manual counter, it simply moves to the next batch safely. An empty list will never cause the program to hang, spin infinitely, or throw an out-of-bounds error.

2. Submission Documentation: Flag Logic

A standard `break` statement will only terminate the loop it is directly nested inside. If we only broke the inner loop upon finding the "STOP" signal, the outer loop would just proceed to the next batch. To fix this, I initialize a `halt_audit` flag to False at the start. When "STOP" triggers, this flag becomes True just before breaking the inner loop. The outer loop immediately checks this flag after the inner loop finishes; if True, it executes a second break on the outer loop.

3. Source Code: anomaly_auditor.py

```python
"""
Author: Rohit Kumar Singh
ID: A26MTAI0004
Project: The Sensor Anomaly Filter
"""

def run_anomaly_filter():
    # Test dataset
    sensor_data_stream = [
        [22.5, 23.0, 22.8],
        [25.1, "ERR", 24.9],
        [30.2, 35.5, 40.1],
        [22.0, 22.1, "STOP"],
    ]

    halt_audit = False

    # Outer loop using range(len())
    for batch_index in range(len(sensor_data_stream)):
        current_batch = sensor_data_stream[batch_index]
        print(f"Auditing Batch {batch_index}: {current_batch}")

        last_reading = None

        # Inner loop checking each value
        for val in current_batch:

            # 1. Emergency stop check
            if val == "STOP":
                print(f"Emergency Shutdown at Batch {batch_index}."
    )
                halt_audit = True
                break

            # 2. Noise/error check
            if val == "ERR":
                print(f"Noise ignored at Batch {batch_index} (ERR).
    ")
                continue

            # 3. Numeric validation before calculations
            if isinstance(val, (int, float)):

                # Threshold breach check
                if val > 35.0:
                    print(f"Anomaly Detected at Batch {batch_index
    }: {val}")

                # Rolling Delta bonus task
                if last_reading is not None:
                    diff_val = round(abs(val - last_reading), 1)
                    if diff_val > 5.0:
```

```
                        print(f"Spike Detected at Batch {batch_index}: "
                              f"{last_reading} -> {val} (Delta {diff_val})")

                # Store valid reading for the next loop's delta check
                last_reading = val

        # Check flag to break outer loop if necessary
        if halt_audit:
            break

    # Triggers only if no 'break' occurred in the outer loop
    else:
        print("Audit Complete: No system-wide failures")


if __name__ == "__main__":
    run_anomaly_filter()
```

anomaly_auditor.py